

Generating SysML v2 Models from Natural Language Requirements Using Large Language Models

Simon Dehn

Institute for Machine Elements and
Systems Engineering (MSE)
RWTH Aachen University
Aachen, Germany
simon.dehn@imse.rwth-aachen.de

Simon Schnürer

Institute for Machine Elements and
Systems Engineering (MSE)
RWTH Aachen University
Aachen, Germany
simon.schnuerer@imse.rwth-aachen.de

Georg Jacobs

Institute for Machine Elements and
Systems Engineering (MSE)
RWTH Aachen University
Aachen, Germany
georg.jacobs@imse.rwth-aachen.de

Gregor Höpfner

Institute for Machine Elements and
Systems Engineering (MSE)
RWTH Aachen University
Aachen, Germany
gregor.hoepfner@imse.rwth-aachen.de

Abstract—The rising complexity of automotive systems demands structured methods for system development. Model-Based Systems Engineering (MBSE) methods offer such a structured approach to manage complexity, but its adoption is hindered by the high effort of manually modelling complex relations in modeling languages such as SysML v2. Information needed to author SysML v2 models is embedded in requirements specifications, which are usually stated in natural language. While Large Language Models (LLMs) are capable of processing natural language to extract information and generate SysML v2 code, output quality varies widely without strong guidance. To address this challenge, this paper presents a structured approach for instructing LLMs to more consistently generate SysML v2 models. We evaluate the robustness of this approach through repeated model generation and compare it against other prompting setups with input of varying structure. Results show that using structured prompting substantially improves generating semantically rich and syntactically coherent SysML v2 models. This work highlights the potential of LLMs to support the adoption of model-based systems engineering by automating the transition from textual requirements to formal SysML v2 models.

Keywords — *Model-Based Systems Engineering, Requirements, Formalization, SysML v2, LLM*

I. INTRODUCTION

The increasing complexity of modern automotive systems demands a shift in product development methods. Traditional engineering methods are reaching their limits as vehicles become more interconnected, software-defined, and functionally dense. In response to these challenges, **Model-Based Systems Engineering (MBSE)** has emerged over the past decade as a promising approach to manage complexity through formalized, model-centric development practices [1].

Despite its potential, the broad implementation of MBSE in industry remains limited. One of the key barriers to adoption is the shortage of qualified personnel and organizational capacity to support the transition from today's predominantly document-centric development processes to future model-

based practices [2, 3]. In practice, requirements specifications at Original Equipment Manufacturers (OEMs) are still mostly unformalized and written in natural language. These textual requirements are available in large quantities and typically contain the information necessary to describe both system architecture and behavior.

This observation highlights an opportunity: instead of treating natural language requirements as a separate artifact from system models, they can be directly leveraged as a valuable input for model creation. With the advent of large language models (LLMs), it becomes possible to automatically process and formalize these requirements into SysML v2 constructs. In this way, requirements that were previously only accessible to engineers through manual interpretation can be transformed into structured representations within a system model.

The proposed approach enables the automatic generation of SysML v2 models from textual specifications that OEMs already produce as part of their standard development processes. By reducing the amount of manual effort needed to translate requirements into formal models, LLMs can significantly lower the entry barrier for MBSE. This not only accelerates the modeling process but also facilitates a wider and quicker adoption of MBSE practices across the industry.

We investigate how LLMs can formalize natural language requirements into SysML v2 models by designing a structured prompt that combines role and goal instructions, an ontology description of system elements and relations, few-shot modeling examples, and a set of requirements. This approach anchors the model's generation process in defined semantics and conventions, reducing interpretive freedom and enabling more accurate transformation of requirements into coherent system models.

The paper is structured as follows: after the **Introduction**, which motivates the need for automated model generation from natural language requirements, the **State of Research** reviews existing work on SysML-based model generation and highlights the gap addressed by this paper. The **Problem Statement** then formulates the research questions arising

Publicly funded within the research project "KIMBA" by the German Federal Ministry of Economic Affairs and Energy (BMWE)

from the challenges of transforming requirement sets into coherent SysML v2 models. The **Methodology** section presents our structured prompting approach, detailing the prompt components and their role in guiding LLM outputs. In the **Evaluation**, we describe the use case of an automotive window opener system, define evaluation criteria, and compare four prompting setups. The **Results** section reports findings on information extraction, syntax, semantics, traceability, runtime, and robustness. A **Discussion** interprets these results, emphasizing the advantages and trade-offs of structured prompting. Finally, the **Conclusion and Outlook** summarize the contributions, limitations, and future research directions before the paper closes with acknowledgments and references.

II. STATE OF RESEARCH

Model-Based Systems Engineering (MBSE) relies on formal models to manage complexity across requirements, structure, and behavior [4]. The Object Management Group's (OMG) modeling language **SysML** (Systems Modeling Language) is a widely adopted standard for formal system modeling [5]. While SysML v1 is a purely graphical modeling language utilizing diagrams to display information, in its latest version **SysML v2**, the language supports **textual notation**, in addition to graphical modeling [6]. The evolution to SysML v2 introduces a textual syntax and clearer formal semantics alongside diagrams, which materially changes automation potential. Natural language artifacts can now be transformed into textual SysML v2 directly, and syntactic validity can be machine-checked. This textual representation is central to current research on "text-to-model" pipelines [7].

A widely adopted concept for systems modeling is the RFLP logic (Requirements, Functional, Logical, Physical). It structures system models into four interlinked views: the Requirements view (R) organizes and manages the system requirements; the Functional view (F) defines the functions necessary to fulfill these requirements; the Logical view (L) translates functions into technological concepts; and the Physical view (P) represents the concrete realization in discipline-specific domains such as mechanics, electronics, or software. These layers are connected by traceability relations, ensuring consistency across abstraction levels and enabling validation from implementation back to requirements. The RFLP logic is considered best practice in industry, especially in automotive and aerospace, as it provides a reference framework for end-to-end Model-Based Systems Engineering (MBSE). [8]

The translation of natural language requirements into formal system models has been advanced by traditional Natural Language Processing (NLP) pipelines that extract entities and relations for SysML v1 diagram generation. For instance, ref. [9] applied rule-based NLP to produce SysML v1.6 diagrams from textual requirements, demonstrating feasibility but limited to graphical representations. The emergence of SysML v2, with its support for textual notation, offers new opportunities for automation. Reference [6] introduces *SysTemp*, a template-driven multi-agent system that generates SysML v2 models, yet it processes limited inputs (1-5 Requirements) and does not address ontological aspects, such as linking system elements with each other for traceability.

Concurrently, Large Language Models (LLMs) have shown promise across diverse Requirements Engineering

(RE) tasks, including elicitation, modeling, validation, specification, prioritization, and traceability. A systematic review (*LLM4RE*) identified that LLMs have notably improved several RE activities, though they also flagged challenges in governance and reliable performance [10]. Another guideline by [11] outlines best practices for selecting and adapting LLMs to specific RE tasks, emphasizing the need for systematic prompt design and task alignment.

In model generation contexts, LLMs have been applied to create Unified Modeling Language (UML) sequence diagrams from natural language requirements [12]. OpenAI's ChatGPT, for example, demonstrated the ability to produce behavioral diagrams that align with stakeholder needs, showing flexibility beyond rule-based systems.

These studies collectively illustrate that LLMs can support the generation of requirement-based SysML v2 artifacts, yet they also highlight limitations in consistency, interpretability, and semantic fidelity.

The presented state-of-the-art methods exhibit limitations:

- Prior research in processing natural language requirements into SysML has focused on **SysML v1 diagrams**, which only achieves visualization and lacks the native textual notation and requirement formalization that enables LLM-driven model generation [9].
- Methods employ classic NLP pipelines rather than leveraging the advanced language understanding and generative capabilities of modern **LLMs** [9, 12].
- Template-based SysML v2 methods (e.g., *SysTemp*) process small input batches (1-5 requirements) and do not address ontological linking between system elements for end-to-end traceability [6].

III. PROBLEM STATEMENT

As outlined in the introduction, the growing complexity of automotive systems calls for more efficient development methodologies. While Model-Based Systems Engineering (MBSE) offers a promising solution, its adoption is hampered by the manual effort and expertise required to create SysML models. One way of supporting this transition is to automatically and systematically extract the necessary information from natural language requirements.

Recent research has demonstrated that LLMs can generate formal models from textual inputs, yet these approaches still face shortcomings. In particular, they often lack semantic correctness, leading to inconsistencies in how requirements are interpreted and represented in models. Moreover, they do not sufficiently ensure ontological consistency, meaning that relations between system elements (e.g., requirement to function, logical, and physical links) are incomplete, ambiguous, or missing, which undermines traceability across abstraction levels. Finally, while SysML v2 offers a textual notation well-suited for automation, generating syntactically correct SysML v2 code remains a challenge for LLMs due to its novelty and formalism.

This paper addresses the open question of how sets of textual requirements can be automatically and systematically transformed into consistent and structured SysML v2 models. Specifically, the following challenges arise:

- Ensuring semantic correctness, so that natural language requirements are accurately formalized into SysML v2 elements without contradictions or misinterpretations.
- Achieving ontological consistency in the generated models, including complete and accurate relations between requirements, functions, logical elements, and physical components to support traceability.
- Guaranteeing syntactic correctness in the SysML v2 output, given that the language is relatively new and not natively supported by most LLMs.

Accordingly, this paper investigates the following research questions:

- **RQ1:** How can the elements of the system architecture be reliably extracted from natural language requirements utilizing LLMs?
- **RQ2:** How can ontological consistency be achieved in the generated models?
- **RQ3:** How can a syntactically correct SysML v2 output be achieved?

By answering these research questions, we aim to understand the current capabilities and limitations of LLM-based approaches for comprehensive SysML v2 model generation.

IV. METHODOLOGY

In contrast to the current state-of-the-research approaches, our work introduces a method that combines the **textual modeling capabilities of SysML v2** with the contextual **semantic understanding of LLMs**. Rather than processing individual requirements in isolation, our approach considers requirement sets, addressing the contextual dependencies that arise in real-world use-cases.

We include the derivation of a hierarchical system architecture, the extraction of system states, attributes, physical parts and handling overlapping references to the same architectural elements. Our approach uses a general-

purpose LLM (OpenAI GPT-5) to extract both system elements and their interrelations, aiming for a flexible and scalable solution.

To investigate how LLMs can be used to formalize natural language requirement sets into a coherent SysML v2 model, we develop a structured prompt that aims to guide the language model toward semantically and syntactically accurate outputs. Four different prompt setups numbered 1 to 4 are evaluated (cf. Fig. 1).

The proposed prompt structure is described in the following paragraphs and consists of a role and goal description supplemented by ontology description, few-shot examples, and lastly, the set of natural language requirements.

In the **Ontology Description**, we define the elements and relations of the system architecture (i.e. the meta-model) that are expected to be extracted from the requirement set. This ontology defines the relevant entities (e.g., *requirements*, *functions*, *logicals*, *parts*) and relationships (e.g., *satisfy*, *perform*, *allocate*), along with rules on modeling for each entity (e.g., how to model *functional requirements* and *performance requirements*). This step ensures that the model has a structured reference framework for interpreting the requirements and relating them to architectural elements.

In the **Few-Shot Examples** part we provide the LLM with a set of three exemplary SysML v2 models of other systems, illustrating how natural language requirements correspond to SysML v2 elements and their relationships as well as how they are expected to be modeled. These examples serve to guide the model's generative behavior and align its output format with the target modeling conventions and SysML v2 syntax.

In the **last part**, the actual Natural Language **Requirements Set** is introduced into the prompt.

With this prompt engineering strategy, we aim to overcome one of the challenges in requirement formalization: By anchoring the model's generation process with an explicit ontology and few shot examples, we provide it with a constrained semantic space and instructive patterns that reduce interpretive freedom. This is particularly important when requirements are complex, interdependent, or span different perspectives, e.g. structure and behavior.

V. EVALUATION

To evaluate the feasibility and effectiveness of our approach, we selected an electric window opener system of a car as a representative use case. This system was chosen due to its cross-domain complexity, modular architecture, and relevance to safety- and function-critical domains in automotive engineering. Moreover, the system offers diverse requirement types, specifying its basic functionality as well as state-based behaviors, appropriate for evaluating the capabilities of large language models in capturing the static system architecture and the system's dynamic behavior.

A. Use Case Description

A SysML v2 system model of the electric window opener is modeled manually and serves as a reference model that the generated models will be compared to. The reference model follows the commonly used RFLP (Requirements, Functional, Logical, Physical) structure. We define the necessary modeling elements (function, logical, and physical) in a dedicated meta-model. The system's behavior is modeled

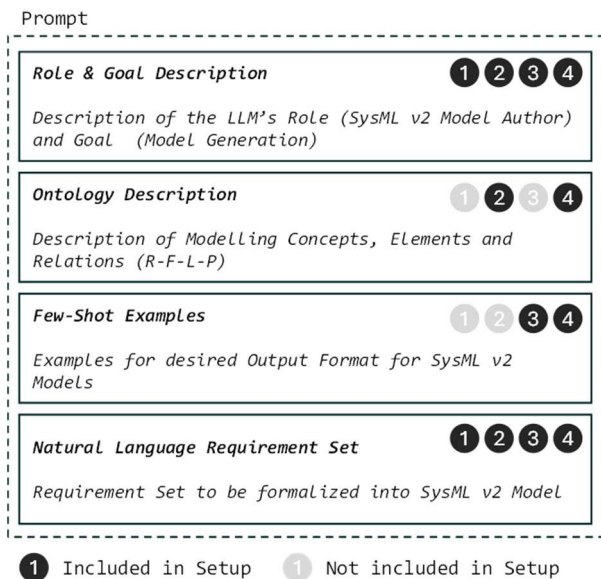


Fig. 1. Structured Prompt for SysML v2 Model Generation from Requirement Sets.

TABLE I. REQUIREMENT SET FOR MODEL GENERATION

ID	Requirement
R1	The system shall move the window between fully open and fully closed positions.
R2	The system shall receive electrical power from the vehicle's power supply for operation.
R3	The system shall be powered by the vehicle's battery.
R4	The system shall be protected by a fuse to prevent electrical overcurrent damage.
R5	The system shall use dedicated wiring to transmit electrical power and control signals between components.
R6	The system shall convert electrical energy into rotational motion using an electric motor.
R7	The system shall convert rotational motion into linear motion to move the window.
R8	The system shall include a gear mechanism to adapt motor torque and speed for window movement.
R9	The window shall be fixed to a mount that transfers motion from the drive mechanism to the glass pane.
R10	The system shall include a linkage to transfer motion from the drive mechanism to the window mount.
R11	The system shall detect and respond to direction commands.
R12	The window shall be guided to prevent tilting or jamming.
R13	The window shall move along guide rails to maintain alignment during operation.
R14	The system shall support manual and automatic modes based on button press duration.
R15	The system shall include a switch to receive manual input from the user.
R16	The system shall include a touch or force sensor to detect contact with obstacles during window movement.
R17	The window shall stop automatically at end positions without overshooting.
R18	The window shall hold its position when no command is active.
R19	The window speed shall remain within 0.5 m/s $\pm$ 10 %.
R20	The system shall start window movement within 200 ms after user input.
R21	The system shall reject false inputs shorter than 20 ms.
R22	The system shall reverse window direction within 50 ms if an obstacle is detected.
R23	The maximum closing force shall not exceed 100 N.
R24	The system shall function correctly with supply voltages from 9 V to 16 V.

using *states* and *transitions*. The model describes basic functions that need to be implemented by the window opener in order to satisfy the requirements: Transforming the incoming electrical power into window movement, accepting user input, controlling the window movement, and ensuring safe operation. The functions are fulfilled by logical concepts, which are finally allocated to physical parts.

1) *Static System Architecture*: In our reference model we use 24 requirements: 18 functional requirements (R1-R18) and 6 performance requirements (R19-R24, cf. TABLE I). The functional requirements contain information about the intended system functions (e.g., “The system shall detect obstacles [...] during window movement.”) and performance requirements specify the model's attributes and desired behavior (e.g., “The maximum closing force shall not exceed 100 N.”). These requirements are formalized and traced to functions, logicals and states using the SysML v2-native requirement element and subject parameter. Performance requirements are modeled as suggested in the examples given.

TABLE II. FUNCTION HIERARCHY AND SATISFY RELATION TO REQUIREMENT

Level	Function	Satisfies
-	moveWindow	R1
I	transformPowerIntoMovement	R2 – R10
II	controlMovement	R11-R13, R19
III	processUserInput	R14,R15
IV	ensureSafety	R16-R18, R22
I.a	regulatePower	R2,R3,R4,R5
I.b	convertToRotationalMotion	R6
I.c	convertToLinearMotion;	R7,R8,R9,R10
II.a	regulateSpeed	R19
II.b	selectDirection,	R11
II.c	guideWindow;	R12,R13
III.a	detectSwitchInput	R15
III.b	interpretPressDuration;	R14
IV.a	detectObstacles	R16
IV.b	stopAtEndPosition	R17
IV.c	holdPosition	R18
IV.d	reverseOnObstacles	R22

by the SysML v2 specification [5]. The system model consists of three hierarchical function levels. The main function moveWindow is decomposed into four subfunctions (I-IV) that are further separated into a total of 12 leaf functions (a-d) on the lowest level of the function hierarchy (cf. TABLE II). The function elements are derived from the entire requirement set and are traced using the relation satisfy.

The *logical* elements of the model are traced to the corresponding functions using the *perform* relation. All Attributes are defined within the logical element, e.g. the attribute *force* is defined inside the logical element *obstacleDetection*. In total, 16 logical elements and 6 attributes are derived from the requirements to cover all functions via the *perform* relation.

The model comprises 16 physical parts and 4 software parts. The physical parts are traced to the logical elements using the *allocate* relation.

The overall model structure with its elements and relations is depicted in Fig. 2, showing the formalization of an exemplary natural language requirement and the traces from *requirement* to *function*, *logical*, and *physical* part.

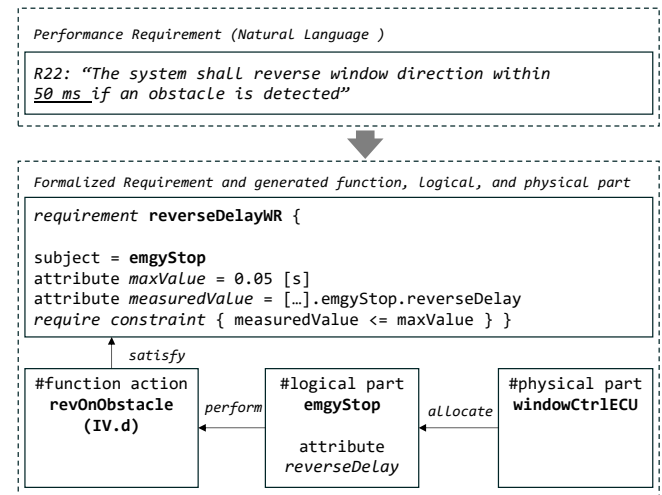


Fig. 2. Formalization of a Requirement and traces to System Elements.

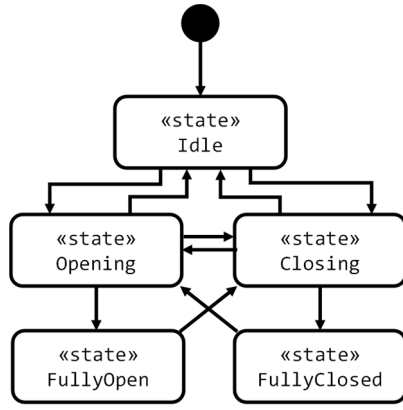


Fig. 3. Dynamic Behavior of the Electric Window Opener.

2) *Dynamic System Behavior*: The dynamic behavior of the window opener system is modeled via the SysML v2 state and transition elements. A total of 5 states and 10 transitions are derived from the requirements (cf. Fig. 3). The requirements are linked to states in which they must be fulfilled by assigning the state to the SysML v2 assume constraint parameter within the requirement. Within the assume constraint parameter the model evaluates if the condition stated in the requirement matches the currently active state of the system element. The operating states in this example can be implicitly derived from the requirement “The system shall move the window between fully open and fully closed positions”.

B. Evaluation

To assess the quality improvements and robustness of our methodology, we conduct a comparative evaluation of **four prompting strategies** with **four evaluation aspects** and are explained in this chapter: *information extraction*, *syntax*, *semantics*, and *traceability*. The monetary cost of carrying out a prompt is measured in *tokens*, which represent the smallest unit of text (e.g. a word, part of a word, or punctuation) that a LLM can process. Costs, in terms of time, are measured by the *runtime* for each run. To evaluate the *robustness* of the output, each of the four prompting strategies is used to generate **three** SysML v2 models. In total, 12 SysML v2 models are created and evaluated with four criteria against the reference model as described in this chapter.

To evaluate the effectiveness of our proposed prompting strategy and to optimize the generated output, four different prompting setups are compared. In the first setup, the LLM is only prompted to generate a comprehensive SysML v2 model (*Role & Goal Description*, cf. Fig. 1) based on the natural language requirements. The second setup additionally includes the *Ontology Description*. Setup 3 includes the *Role & Goal Description*, three *Few-Shot Examples* and the *Requirements Set*. Finally, setup 4 includes all the components depicted in Fig. 1.

1) *Information Extraction*: This criterion evaluates the model elements (i.e. functions, logicals, parts, relations, and states) that are extracted by the LLM from the given requirement set. The extracted model elements are compared against a list of manually retrieved model elements that are derived from the requirements directly. As metrics we chose recall (1), precision (2), and F1 (3). In this case, a True Positive is a model element that is generated by the LLM and

exists in the reference model. A False Positive is a model element that was generated by the LLM but doesn't exist in the reference model, and a False Negative is a model element that was not generated but exists in the reference model.

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives} \quad (1)$$

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad (2)$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (3)$$

2) *Traceability*: This criterion evaluates the coverage and correctness of trace links in the generated model. Traceability is measured along three relation types that connect the requirement, functional, logical, and physical layers:

- R–F (Requirement to Function): captured through the satisfy relation, linking each leaf function to the corresponding occurrence within the requirements.
- F–L (Function to Logical): captured through the perform relation, binding logical parts to the functions they realize.
- L–P (Logical to Physical): captured through the allocation relation, mapping physical parts to logical elements.

For each category, the percentage of correctly established links is calculated as coverage (4)

$$Coverage = \frac{Correct\ Traces}{Expected\ Traces} \cdot 100\% \quad (4)$$

The coverage values are reported separately for R–F, F–L, and L–P. This criterion provides a quantitative measure of how completely the generated model maintains end-to-end traceability across abstraction levels.

3) *Syntax and Semantics*: The *Syntax* criterion evaluates the correctness of the generated SysML v2 code. The syntax of the code is checked by using a SysML v2 parser. A pass indicates a 100% syntactically correct code, fail indicates an incorrect code. The *Semantics* criterion evaluates the methodological correctness. It verifies whether the generated model follows the same modeling principles, naming conventions, and structuring rules as the reference. The evaluation is structured along three methodological dimensions (a-c). Each dimension is rated qualitatively as fully met, partly met, or not met:

a) *Requirement Formalization*: Checks whether quantitative requirements (R19-R24, cf. TABLE I) are identified correctly as Performance Requirements, bound to logical leaf attributes with appropriate ISQ/SI quantities, and without initial values. Incorrect typing, misplaced numeric values, or missing bindings lower the fulfillment level. Functional Requirements: Checks whether each functional requirement (R1-R18, cf. TABLE I) is satisfied by a leaf function that matches the requirement's subject. Using other relation types (e.g., refine instead of satisfy) or satisfying functional requirements at non-leaf functions reduces fulfillment.

b) *Architecture Modeling*: Checks whether verb-object naming is applied, whether satisfy relations are placed on leaf functions, whether perform is used to bind logicals to functions, and whether allocations map logicals to physicals. Deviations from these patterns lower the fulfillment level.

c) *Behavioral Modeling*: Checks whether states are defined with entry actions assigning the system's active state, and whether transitions between Idle, operative, and terminal states are plausible and complete. Missing states, pseudocode-only representations, or absence of entry assignments reduce fulfillment.

4) *Robustness Assessment and Operational Metrics*: For robustness assessment, we execute the model generation process three times for each of the four prompting setups. This repetition allows us to observe potential variance in the model outputs due to the non-deterministic nature of LLMs. The resulting models are evaluated and compared to each other with respect to the applied strategy. In addition to robustness, we also evaluate the operational metrics of the LLM request, namely token usage and runtime. Runtime refers to the time required by the model to generate a response, while tokens represent the smallest units of text (words, subwords, or punctuation) that the LLM processes and that directly determine computational cost. These metrics provide insights into the efficiency and cost implications of each prompting setup.

VI. RESULTS

All results reported in this section were generated with OpenAI GPT-5. We generated 12 SysML v2 models from the electric window opener requirements with four prompting setups and three runs each. Each model was compared against the manually authored reference model. For evaluation we used the criteria defined in the Evaluation section: information extraction (precision/recall/F1), traceability coverage, and syntax and semantics. All results are indicated in their average score over the three runs that were carried out. The variance of the results between the runs is stated in the text. A lower variance is better, as it is an indicator for more robust outputs.

1) *Information Extraction*: TABLE III shows the results for information extraction. The average F1-score steadily improves with increasing prompt structure from setup 1 (86%) over setup 2 (95%) and setup 3 (96%) to setup 4 (98%). As precision is 100% in every setup, the gains in F1-score come from higher recall (76%-95%). In terms of robustness, the run-to-run variance in recall is smallest in setups 1 (74-79%) and 3 (90-92%), larger in setup 4 (92-97%), and largest in setup 2 (85-97%).

TABLE III. AVERAGE RESULTS FOR RECALL, PRECISION, AND F1 FOR INFORMATION EXTRACTION

Setup	Recall [%]	Precision [%]	F1 [%]
1	76	100	86
2	91	100	95
3	91	100	96
4	95	100	98

TABLE IV. AVERAGE RESULTS FOR TRACEABILITY COVERAGE

Setup	R-F [%]	F-L [%]	L-P [%]
1	33	67	91
2	99	100	100
3	100	65	85
4	100	100	95

2) *Traceability*: The average traceability coverage results are shown in TABLE IV. setups 2 and 4 deliver near-complete coverage on average, whereas setup 3 is limited by unstable F-L links (runs: 93/30/71%). Per-run inspection shows the main outliers are F-L and, to a lesser extent, L-P in setups 3 and 4. R-F is uniformly > 96% in every run except setup 1.

TABLE V. AVERAGE RESULTS FOR TRACEABILITY

Setup	Syntax	Semantics	Token [#]	Runtime [s]
1	Fail	Not met	10,204	118
2	Fail	Partly met	14,879	226
3	Pass	Partly met	18,349	200
4	Pass	Fully met	23,811	343

3) *Syntax and Semantics*: Results for syntax and semantics are shown in TABLE V. Only setups 3 and 4 consistently pass the SysML v2 parser. Setups 1 and 2 fail in all runs. Semantic conformity follows the same trend: Setup 1 not met, setups 2 and 3 partly met, setup 4 fully met. Results were consistent in all three runs.

4) *Robustness Assessment and Operational Metrics*: Across the three runs per setup, the results show clear robustness patterns. For information extraction, precision stays 100% in every run. Recall is high and stable in setups 3 and 4, low but steady in setup 1, and high yet volatile in setup 2. For traceability, R-F becomes robust as soon as prompt structure is added (setups 2-4). Particularly in setup 3, the F-L relation shows the most variance while L-P remains generally high with only minor dips, for example 86% in setup 4, run 2. Finally, syntactic and semantic consistency are achieved only when few-shot examples are included, i.e., in setups 3 and 4. The increasing prompt length increases token usage. The average number of tokens rise from 10,204 (setup 1) to 23,811 (setup 4). As of August 2025, one run with setup 4 costs approximately 0,15 \$. Average runtime increases from 118 s to 343 s over setups 1 to 4. The longest single run is 521 s in setup 4.

VII. DISCUSSION

The results show that adding structure to the prompt changes both the quality and the stability of the pipeline. Introducing only the ontology in setup 2 increases recall substantially compared to the minimal prompt (setup 1) and yields nearly complete traceability across R-F, F-L, and L-P on average. At the same time, this setup still fails on syntax and only partially meets semantics, and it shows the largest variance in recall across runs. The ontology helps the model decide what to extract and how items should relate, but it does

not sufficiently constrain how to express the model in valid SysML v2.

Adding few-shot examples (setup 3) has a different effect. The results of this setup reliably pass parser checks and move semantics from “not met” to “partially met.” However, without the ontology, bindings between functions and logical elements become unstable, which reduces F-L coverage despite full R-F coverage. Few-shot examples achieve correct SysML v2 syntax, but they do not fully capture architectural binding rules.

The strategy in setup 4 combines the advantages of setups 2 and 3. It preserves the syntactic and semantic benefits of few-shot examples while retaining the architectural discipline of using an ontology. This setup delivers the best overall performance, with the highest information extraction F1 and near complete traceability. The remaining variability is concentrated in L-P, which is consistent with the inherent difficulty of mapping logical abstractions to concrete parts when the requirements do not spell out implementation choices. Another advantage of this method is its adaptability. The ontology description and few shot examples can be extended or adapted for different application domains, system classes or methodological concepts. In future applications, this flexibility could enable the deployment of the method across a range of engineering projects with reduced manual overhead. With this approach we move closer to a fully automated pipeline for requirements-to-model translation, which aligns with the vision of scalable MBSE.

Quality gains come with higher token counts and runtimes. Tokens and runtime roughly doubles to triples from setup 1 to setup 4. For practical engineering workflows, a less resource intensive approach might be to use the structured prompt to produce a high-fidelity baseline, then focus human review on the small set of binding decisions rather than repeatedly regenerating the full model.

VIII. CONCLUSION AND OUTLOOK

The present study evaluated the use of large language models for generating SysML v2 models from requirement sets. Four prompting setups of increasing complexity were compared. Setup 1 only included a role and goal description, setup 2 was supplemented with an ontology description, setup 3 contained few-shot examples, and setup 4 had both, ontology description, and few-shot examples. The results show that adding structured information such as ontology description and few-shot examples to the prompt is the key to achieving syntactically correct SysML v2 models with high traceability coverage and strong information extraction. Minimal prompting as in setup 1 underperforms on recall and produces unstable traces, whereas the combination of an ontology and few-shot examples improves both quality and stability. Remaining errors are mainly in logical-to-physical allocations, and the quality gains come with higher token usage and longer runtimes. The results show that the capabilities and limitations of LLM-based model generation can be framed along the three research questions:

- **RQ1:** LLMs extract system elements with high precision (100%) and high recall. Recall increases with ontology description and few-shot examples.
- **RQ2:** The ontology significantly improves architectural completeness and traceability but does

not guarantee correct SysML v2 realization. Combining it with few-shot examples is necessary.

- **RQ3:** Syntactically correct SysML v2 output was achieved when few-shot examples were included in the prompt (setup 3, setup 4), confirming that few-shot examples are essential to robustly meet textual modeling conventions.

Outputs are robust in precision but still variable in recall (notably setup 2) and in traceability bindings, especially F-L (setup 3) and occasionally L-P (setup 4). The combined prompt (setup 4) is the most reliable overall, with volatility limited to allocations.

Structured prompting is necessary to reach the combination of syntactic correctness, semantic conformity, and high traceability, which comes with higher token counts and runtimes. The best setup (setup 4) achieves near-complete coverage and the highest F1-score, with small remaining variance concentrated in logical-to-physical mappings.

Future work should extend this evaluation to industrial-scale, heterogeneous requirement data and investigate methods for stabilizing outputs, such as post-processing or human-in-the-loop feedback. Integration into MBSE toolchains will be essential to assess the practical value of LLM-based model generation in productive engineering environments.

REFERENCES

- [1] T. Zerwas *et al.*, "Mechanical concept development using principle solution models," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 1097, no. 1, p. 12001, 2021, doi: 10.1088/1757-899X/1097/1/012001.
- [2] K. Mokus and P. Katsaros, "A survey on the formalisation of system requirements and their validation," *Array*, vol. 7, p. 100030, 2020, doi: 10.1016/j.array.2020.100030.
- [3] K. Henderson, T. McDermott, and A. Salado, "MBSE adoption experiences in organizations: Lessons learned," *Systems Engineering*, vol. 27, no. 1, pp. 214–239, 2024, doi: 10.1002/sys.21717.
- [4] G. Jacobs, C. Konrad, J. Bertho, T. Zerwas, G. Höpfner, and K. Spitz, "Function-Oriented Model-Based Product Development," in *Design Methodology for Future Products*, D. Krause and E. Heyden, Eds., Cham: Springer International Publishing, 2022, pp. 243–263.
- [5] OMG, *OMG Systems Modeling Language*, 2024. Accessed: Aug. 27 2024. [Online]. Available: <https://www.omg.org/spec/SysML/2.0/Beta2/Language/PDF>
- [6] Y. Bouamra, B. Yun, A. Poisson, and F. Armetta, "SysTemp: A Multi-Agent System for Template-Based Generation of SysML v2," Jun. 2025. [Online]. Available: <http://arxiv.org/pdf/2506.21608>
- [7] E. Cibrián, J. Olivert-Iserte, J. Llorens, and J. M. Álvarez-Rodríguez, "An agent-based approach for the automatic generation of valid SysMLv2 Models in industrial contexts," *Computers in Industry*, vol. 172, p. 104350, 2025, doi: 10.1016/j.compind.2025.104350.
- [8] J. Krog, C. Akbas, B. Nolte, and T. Vietor, "Development of a Functional and Logical Reference System Architecture in Automotive Engineering," *Systems*, vol. 13, no. 3, p. 141, 2025, doi: 10.3390/systems13030141.
- [9] S. Zhong, A. Scarinci, and A. Cicirello, "Natural Language Processing for systems engineering: Automatic generation of Systems Modelling Language diagrams," *Knowledge-Based Systems*, vol. 259, p. 110071, 2023, doi: 10.1016/j.knosys.2022.110071.
- [10] J. A. Khan, S. Qayyum, and H. S. Dar, *Large Language Model for Requirements Engineering: A Systematic Literature Review*, 2025.
- [11] A. Vogelsang and J. Fischbach, "Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline," 2024. [Online]. Available: <https://arxiv.org/pdf/2402.13823>
- [12] A. Ferrari, S. Abualhaija, and C. Arora, "Model Generation with LLMs: From Requirements to UML Sequence Diagrams," Sep. 2024. [Online]. Available: <http://arxiv.org/pdf/2404.06371v2>